20. Gün — CrewAI Baştan Sona

1. CrewAI nedir?

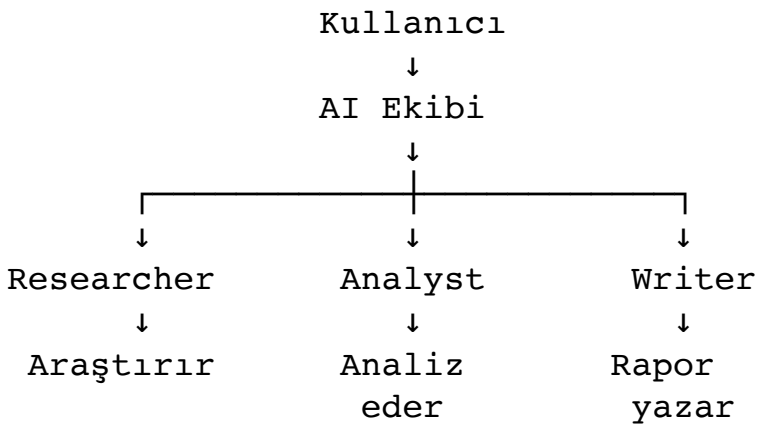
CrewAI, **birden fazla yapay zeka ajanını bir ekip gibi birlikte çalıştırmak için kullanılan açık kaynaklı bir multi-agent framework'tür.**

Normalde tek bir LLM'e şöyle diyebiliriz:

```
"Bu şirketi araştır, verilerini analiz et,
rapor yaz ve sonucu kontrol et."
```

Burada tek model bütün işi yapmaya çalışır.

CrewAI yaklaşımında ise işi uzmanlara böleriz:



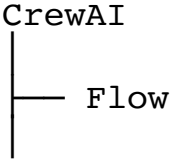
Yani gerçek bir şirkette nasıl farklı çalışanların farklı görevleri varsa, CrewAI'de de farklı **Agent'lar** oluşturuyoruz.

Temel amaç:

Karmaşık bir problemi tek bir AI'ya yüklemek yerine, uzmanlaşmış AI agent'larının birlikte çözmesini sağlamak.

2. CrewAI'nin temel mimarisi

CrewAI'yi anlamak için önce iki büyük yapıyı ayırmamız gerekiyor:



```
└── Crew
```

Bunların görevleri farklıdır.

3. Flow nedir?

Flow, uygulamanın genel iş akışını yöneten yapıdır.

Yani:

"Sistem hangi sırayla ilerleyecek?"

sorusunu Flow cevaplar.

Örneğin:

```
START
  ↓
Kullanıcı sorusunu al
  ↓
Soruyu analiz et
  ↓
Araştırma gerekiyor mu?
   │
   ├── Hayır → Normal cevap oluştur
   │
   └── Evet
         ↓
    Research Crew
         ↓
    Sonucu kontrol et
         ↓
    Veritabanına kaydet
         ↓
        END
```

Flow içerisinde:

- koşullar,
- dallanmalar,
- event'ler,
- state,
- döngüler,
- Python kodları

bulunabilir.

State nedir?

Flow'un işlem boyunca taşıdığı bilgidir.

Örneğin:

```
Kullanıcının sorusu
Araştırma sonucu
Onay durumu
Final cevap
```

Flow bunları adımlar arasında saklayabilir.

Event-driven çalışma

Bir olay gerçekleştiğinde başka bir işlem tetiklenebilir.

```
Araştırma tamamlandı
        ↓
"research_completed" event
        ↓
Rapor oluştur
```

Bu yüzden Flow'a:

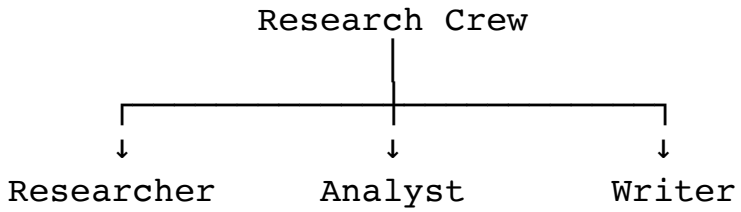
Uygulamanın omurgası

diyebiliriz.

4. Crew nedir?

Crew, belirli bir görevi çözmek için birlikte çalışan agent ekibidir.

Örneğin:



Flow:

"Araştırma yapılması gerekiyor."

der.

Sonra işi Crew'e verir.

Crew içerisindeki agent'lar kendi görevlerini gerçekleştirir.

Sonuç Crew'den tekrar Flow'a döner.

Tam akış:

```
FLOW
 ↓
Karmaşık görev
 ↓
CREW
 ↓
Agent'lar çalışır
 ↓
Crew sonucu
 ↓
FLOW
 ↓
Sonraki işlem
```

Bu nedenle:

Flow = süreci yönetir.

Crew = karmaşık AI işini yapar.

5. Flow mu Crew mü?

Aslında çoğu gerçek uygulamada cevap:

İkisi birlikte.

Basit, kesin ve tahmin edilebilir işleri kod ile yönetmek mantıklıdır.

```
FLOW
│
├── if
├── loop
├── API
├── database
└── normal Python
```

Ama:

- araştırma,
- karar verme,
- değerlendirme,
- içerik üretme,
- karmaşık problem çözme

gibi işler geldiğinde:

```
FLOW
  ↓
CREW
```

kullanabiliriz.

Bu bizim daha önce konuştuğumuz:

Code Orchestration + LLM Orchestration

ayrımına çok benziyor.

6. Crew'ün içinde neler var?

Bir Crew'ün temel yapısını şöyle düşünebiliriz:

```
CREW
 │
 ├── Agent
 ├── Task
 ├── Tool
 ├── Context
 ├── Process
 ├── Memory
 └── LLM
```

Bunların hepsini tek tek görelim.

7. Agent nedir?

Agent, **bir görevi gerçekleştiren yapay zeka çalışanıdır.**

Örneğin:

```
Researcher Agent
```

Bu agent'a:

- kim olduğunu,
- amacını,
- uzmanlığını

söyleriz.

CrewAI'de özellikle üç kavram vardır:

```
role
goal
backstory
```

Role

Agent'ın rolü.

```
Senior AI Researcher
```

Yani:

"Sen kimsin?"

Goal

Agent'ın amacı.

```
Find accurate information about AI Agent frameworks.
```

Yani:

"Neyi başarmalısın?"

Backstory

Agent'ın geçmişini/uzmanlığını tanımlar.

```
You are an experienced AI researcher
specialized in multi-agent systems.
```

Bunun amacı agent'ın nasıl davranması gerektiğini daha iyi belirlemektir.

Sonuç olarak:

```
Agent
=
LLM
+
Role
+
Goal
+
Backstory
+
Tools
+
Memory vb.
```

8. Task nedir?

Agent'ın **kim olduğunu** tanımladık.

Şimdi ona bir iş vermemiz gerekiyor.

Bu işe **Task** denir.

Örneğin:

```
Agent:
AI Researcher

Task:
CrewAI'nin son özelliklerini araştır.
```

Buradaki önemli ayrım:

Agent = işi kim yapacak?

Task = hangi iş yapılacak?

Task içerisinde özellikle iki alan önemlidir:

```
description
expected_output
```

Description

Görevin ne olduğunu açıklar.

```
Research the latest developments in AI agents.
```

Expected Output

Agent'tan nasıl bir sonuç istediğimizi belirtir.

```
A detailed report containing five important developments.
```

Bu çok önemlidir çünkü sadece:

"Araştır."

demek yerine agent'a çıktının nasıl olması gerektiğini de söylüyoruz.

9. Agent ile Task farkı

Çok net:

```
AGENT
↓
Çalışan

TASK
↓
Çalışanın yapacağı iş
```

Gerçek hayat örneği:

```
Çalışan:
Finans Analisti

Görev:
NVIDIA'nın finansal sonuçlarını analiz et.
```

CrewAI karşılığı:

```
Agent:
Financial Analyst
```

```
Task:
Analyze NVIDIA financial results.
```

10. Tools nedir?

LLM kendi başına dış dünyaya erişemez.

Ama agent'a **Tool** verirsek dış sistemlerle işlem yapabilir.

```
Agent
  │
  ├── Google Search
  ├── API
  ├── Database
  ├── GitHub
  ├── Python
  └── Custom Tool
```

Örneğin:

```
Researcher Agent
      ↓
Search Tool
      ↓
Internet
      ↓
Bilgi
      ↓
Agent
```

Böylece agent sadece modelin eğitim bilgisini kullanmaz.

Gerçek sistemlerden veri alabilir.

11. Hazır Tool ve Custom Tool

İki yaklaşım vardır.

Hazır Tool

Framework'te bulunan veya başka kütüphanelerden aldığımız araçlar.

Örneğin:

```
GitHub Search
YouTube Search
Web Search
JSON Search
```

Custom Tool

Kendi ihtiyacımız için yazdığımız araç.

Mesela bizim Digital Twin projesinde:

```
get_github_projects(username)
```

bir tool olabilir.

Akış:

```
Career Agent
     ↓
GitHub Tool
     ↓
GitHub API
     ↓
Repo verileri
     ↓
Career Agent
```

Bu nedenle agent'ların gücü sadece LLM'den gelmez.

Agent'ın gerçek gücü = LLM + Tools

12. Context nedir?

Bir agent'ın ürettiği sonuç başka bir agent tarafından kullanılabilir.

Örneğin:

```
Researcher
    ↓
Araştırma sonucu
    ↓
```

```
CONTEXT
    ↓
Writer
    ↓
Makale
```

Researcher'ın çıktısı Writer için context olur.

Örneğin:

Task 1

```
2026 yılındaki Agentic AI gelişmelerini araştır.
```

Çıktı:

```
Araştırma Raporu
```

Task 2

```
Bu araştırmayı kullanarak blog yazısı oluştur.
```

Writer sıfırdan araştırmak zorunda kalmaz.

Önceki görevin sonucunu kullanır.

13. YAML nedir ve neden kullanıyoruz?

CrewAI projelerinde Agent ve Task tanımlarını Python kodunun içine yazmak zorunda değiliz.

Bunları YAML dosyalarında tutabiliriz.

Genelde:

```
config/
│
├── agents.yaml
└── tasks.yaml
```

olur.

agents.yaml

Kimlerin çalışacağını tanımlar.

```
researcher:
  role: >
    Senior AI Researcher

  goal: >
    Research artificial intelligence topics

  backstory: >
    You are an experienced AI researcher.
```

tasks.yaml

Ne yapılacağını tanımlar.

```
research_task:
  description: >
    Research the latest AI Agent developments.

  expected_output: >
    A detailed research report.
```

Bu sayede:

```
agents.yaml
↓
KİM?

tasks.yaml
↓
NE YAPACAK?
```

oluyor.

14. Neden YAML kullanmak avantajlı?

Çünkü agent tanımlarını uygulama kodundan ayırıyoruz.

```
YAML
↓
```

```
Configuration

Python
↓
Application Logic
```

Örneğin Researcher'ın rolünü değiştirmek istiyorsak bütün Python kodunu değiştirmek yerine YAML'ı değiştirebiliriz.

Bu özellikle büyük multi-agent projelerinde sistemi çok daha düzenli hale getirir.

15. `crew.py` nedir?

Şimdi:

```
agents.yaml
tasks.yaml
```

hazır.

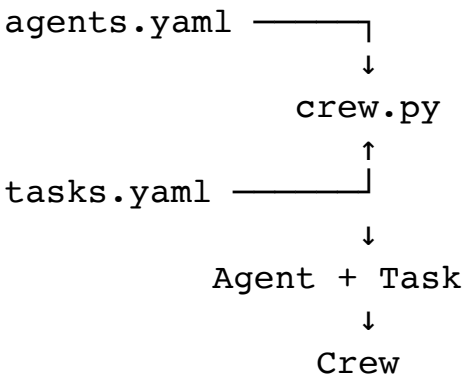
Ama bunların çalışan CrewAI Agent ve Task nesnelerine dönüştürülmesi gerekiyor.

Burada:

```
crew.py
```

devreye giriyor.

Mantık:



Yani:

`crew.py`, ekibin nasıl oluşturulacağını tanımlar.

Burada:

- Agent'lar,
- Task'lar,
- Tools,
- Crew,
- Process

birbirine bağlanır.

16. `main.py` nedir?

`crew.py` sistemi **tanımlar**.

`main.py` ise sistemi **başlatır**.

Örneğin:

```
inputs = {
    "topic": "Artificial Intelligence"
}
```

gibi girdiler hazırlanabilir.

Sonra Crew çalıştırılır.

```
main.py
 ↓
Inputs
 ↓
Crew
 ↓
Tasks
 ↓
Agents
 ↓
Result
```

Bu ayrımı unutma:

crew.py → sistemi kurar.

main.py → sistemi çalıştırır.

17. İlk CrewAI projesi

CrewAI'nin CLI'ı ile proje oluşturabiliriz.

```
crewai create crew
```

Bu bize hazır proje yapısı oluşturur.

Kabaca:

```
project/
│
├── src/
│   └── project/
│
│       ├── config/
│       │   ├── agents.yaml
│       │   └── tasks.yaml
│       │
│       ├── crew.py
│       └── main.py
│
└── pyproject.toml
```

Sonra projeyi:

```
crewai run
```

ile çalıştırabiliriz.

Tam mantık:

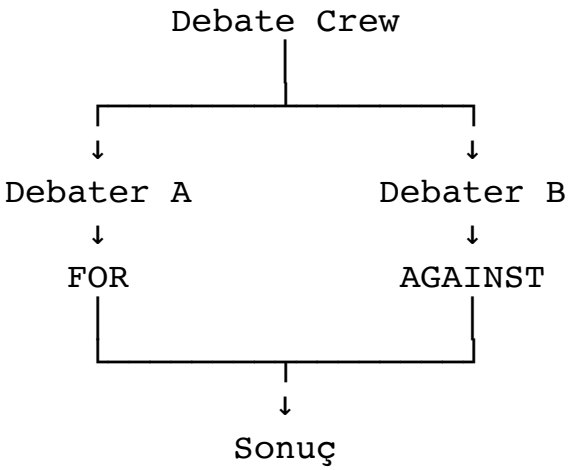
```
crewai create crew
        ↓
Proje oluşur
        ↓
agents.yaml
        ↓
tasks.yaml
        ↓
crew.py
        ↓
main.py
        ↓
crewai run
        ↓
```

```
Crew çalışır
```

18. Debate Crew

İlk örneğimiz **Debate Crew**.

Burada iki agent farklı görüşleri savunabilir.



Örneğin konu:

```
"Artificial intelligence will improve education."
```

Agent A:

```
FOR
```

savunur.

Agent B:

```
AGAINST
```

savunur.

Burada asıl öğrendiğimiz şey tartışma değil.

Asıl öğrenilen:

```
Birden fazla Agent
+
```

```
Birden fazla Task
+
Farklı Roller
+
Crew
```

mantığıdır.

19. Process nedir?

Şimdi Crew'ü kurduk.

Fakat agent'ların görevleri **hangi düzenle yapacağını** belirlememiz gerekiyor.

Buna:

Process

diyoruz.

CrewAI'de bizim için özellikle iki önemli yaklaşım var:

```
Sequential
Hierarchical
```

20. Sequential Process

Sequential = görevlerin sırayla yapılması.

```
Task 1
 ↓
Task 2
 ↓
Task 3
 ↓
Result
```

Örneğin:

```
Research
   ↓
Analysis
```

```
    ↓
Writing
```

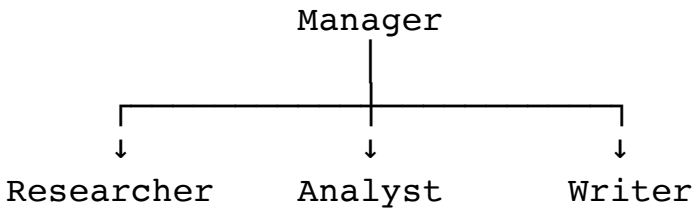
Research tamamlanmadan Analysis başlamaz.

Analysis tamamlanmadan Writing başlamaz.

Bu klasik pipeline/workflow yapısına benzer.

21. Hierarchical Process

Burada sistemde bir **Manager** vardır.



Manager:

- görevi analiz eder,
- kime vereceğini belirler,
- agent'lara görev dağıtır,
- sonuçları kontrol eder,
- gerekirse yeniden görev verir.

Bu, daha önce öğrendiğimiz:

```
Manager–Worker
Supervisor Pattern
```

mantığıdır.

Sequential'da sıra büyük ölçüde bellidir.

Hierarchical'da manager daha fazla karar verir.

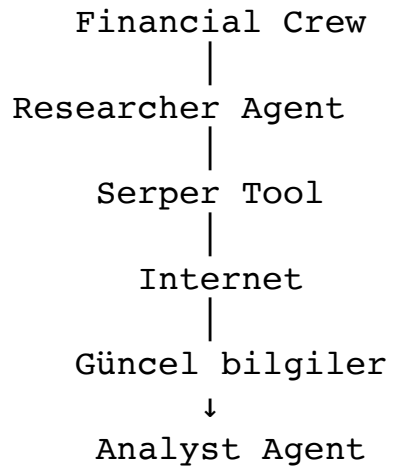
22. Day 2 — Tools ve Context

Buradan sonra CrewAI'nin temel yapısını öğrendik.

Şimdi agent'lara dış dünya yetenekleri vereceğiz.

Financial Research Crew yapacağız.

Örneğin:



Burada iki yeni kavram özellikle kullanılacak:

```
Tools
Context
```

23. Serper Tool

Serper, arama yapmamızı sağlayan bir araç olarak kullanılacak.

Örneğin kullanıcı:

```
"Tesla'nın güncel durumunu araştır."
```

diyor.

Agent:

```
Financial Researcher
        ↓
    Serper Tool
        ↓
   Search Engine
        ↓
   Güncel sonuçlar
        ↓
     Researcher
```

Böylece agent güncel bilgi elde edebilir.

24. CrewAI projesi oluşturmanın temel mantığı

Bir CrewAI projesini aslında beş ana aşamada düşünebiliriz:

```
1. Agent'ları tanımla
        ↓
2. Task'ları tanımla
        ↓
3. Tools / Context ekle
        ↓
4. Crew ve Process oluştur
        ↓
5. Crew'ü çalıştır
```

Bütün projeler temelde bunun farklı versiyonlarıdır.

25. Day 3 — Structured Outputs

Agent'ın çıktısı normalde serbest metindir.

Örneğin:

```
I believe NVIDIA is the best company because...
```

Ama yazılım açısından bu bazen kötü.

Çünkü Python'ın bilgiyi ayıklaması gerekir.

Bunun yerine:

```
{
  "company": "NVIDIA",
  "score": 9,
  "reason": "Strong AI growth",
  "approved": true
}
```

gibi düzenli bir çıktı isteriz.

Buna:

Structured Output

diyoruz.

26. Pydantic Structured Output

Python'da çıktının yapısını Pydantic ile tanımlayabiliriz.

Mantıken:

```
LLM
 ↓
Pydantic Model
 ↓
Validation
 ↓
Structured Data
```

Örneğin:

```
class StockSelection(BaseModel):
    company: str
    score: int
    reason: str
```

Agent artık istediği biçimde uzun bir paragraf vermek yerine bu yapıya uyan veri üretmeye çalışır.

Bu özellikle:

- API,
- database,
- başka agent,
- guardrail,
- Python kodu

ile çalışırken çok önemlidir.

27. Stock Picker Crew

Day 3'te **Stock Picker Crew** oluşturacağız.

Burada:

```
Stock Picker Crew
```

```
│
├── Research
├── Analysis
├── Custom Tools
├── Context
├── Structured Output
├── Memory
└── Hierarchical Process
```

gibi önceki kavramlar birleşmeye başlayacak.

28. Custom Tools

Hazır araçların yetmediği yerde kendi tool'umuzu oluşturabiliriz.

Örneğin:

```
get_stock_price()
```

Agent:

```
Stock Agent
    ↓
get_stock_price()
    ↓
Stock API
    ↓
Price
```

Custom Tool'da açıklama çok önemlidir.

Çünkü LLM:

"Bu tool ne işe yarıyor ve ne zaman kullanmalıyım?"

sorusunun cevabını tool açıklamasından anlamaya çalışır.

29. Memory

Memory, agent sisteminin geçmiş bilgileri saklamasına yardımcı olur.

Örneğin:

```
İlk işlem
 ↓
Apple analiz edildi
 ↓
Memory
 ↓
Yeni işlem
 ↓
Agent önceki bilgiyi hatırlayabilir
```

Burada Context ile Memory'yi karıştırmayalım.

Context

Şu anki görev için verilen bilgi.

```
Research sonucu → Writer
```

Memory

Geçmiş işlemlerden saklanan bilgi.

```
Önceki konuşmalar / çalışmalar
        ↓
Sonraki işlemlerde kullanılır
```

Kısaca:

Context = mevcut işin bilgisi.

Memory = geçmişten gelen bilgi.

30. Day 4 — Coding Agent

Şimdi agent sadece araştırma ve metin üretmeyecek.

Kod da yazacak.

```
User
 ↓
Coding Agent
 ↓
```

```
Kod üret
 ↓
Kodu çalıştır
 ↓
Sonucu incele
 ↓
Gerekirse düzelt
```

Bu daha agentic bir davranıştır.

Agent sadece:

"İşte sana Python kodu."

demiyor.

Kodun gerçekten çalışıp çalışmadığını da test edebiliyor.

31. Buradaki güvenlik problemi

LLM'in yazdığı kodu doğrudan kendi bilgisayarımızda çalıştırmak tehlikeli olabilir.

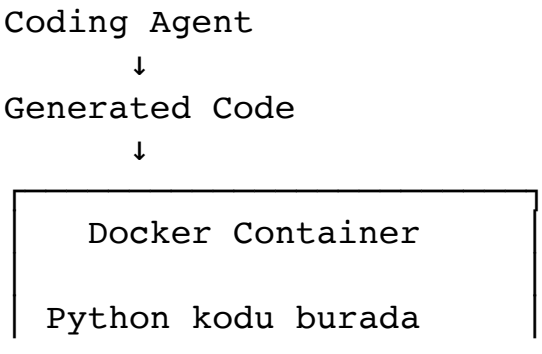
Örneğin yanlışlıkla:

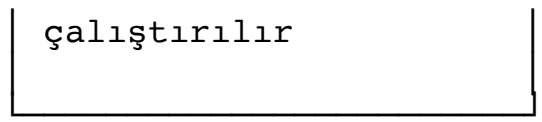
```
Dosya silebilir
Sisteme zarar verebilir
Yanlış komut çalıştırabilir
Bağımlılıkları bozabilir
```

Bu nedenle kodu izole bir ortamda çalıştırmak isteriz.

32. Docker Sandbox

Burada daha önce öğrendiğimiz Docker devreye giriyor.






```
      ↓
Execution Result
      ↓
Coding Agent
```

Container host sistemden izole olduğu için daha kontrollü bir ortam elde ederiz.

Yani:

Sandbox = agent'ın ürettiği kodu ana sistemden ayrı ve kontrollü bir ortamda çalıştırmak.

33. Sandbox Tool

Agent'ın Docker'a erişebilmesi için ona bir tool veririz.

```
Coding Agent
      ↓
Sandbox Tool
      ↓
Docker Container
      ↓
Code Execution
      ↓
Result
```

Yine temel agent mantığına geldik:

```
Agent
 ↓
Tool
 ↓
External System
```

34. UV ve requirements

Kod yazmak yetmez.

Kodun paketlere ihtiyacı olabilir:

```
import pandas
import requests
```

Bunların kurulması gerekir.

Burada:

```
uv
requirements
```

kullanılabilir.

Akış:

```
Agent kod oluşturur
        ↓
Gerekli paketleri belirler
        ↓
uv
        ↓
Sandbox environment
        ↓
Dependencies kurulur
        ↓
Kod çalıştırılır
```

35. Day 5 — Multi-Agent Engineering Team

Son aşamada artık oyuncak örneklerden daha gerçekçi bir sisteme geçiyoruz.

Bir **AI Engineering Team** kuruyoruz.

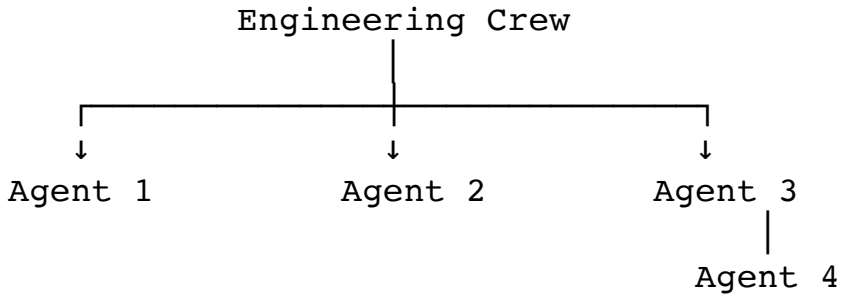
Toplam dört agent olacak.

Her agent'ın farklı:

```
Role
Goal
Backstory
Task
Tools
LLM
```

özellikleri olabilir.

Mantıken:



Bunlar birlikte bir yazılım geliştirme sürecini gerçekleştirebilir.

36. MCP burada nereye giriyor?

MCP'yi daha önce gördük.

MCP bir multi-agent framework değildir.

Model Context Protocol, agent'ların dış araçlara ve veri kaynaklarına standart şekilde bağlanmasını sağlar.

```
CrewAI Agent
      ↓
      MCP
      ↓
External Service
```

Örneğin Day 5'te:

```
Engineering Agent
       ↓
Context7 MCP
       ↓
Güncel dokümantasyon
       ↓
Engineering Agent
```

Agent dış kaynaktan ihtiyaç duyduğu bilgiyi alabilir.

Bu bizim Digital Twin'deki:

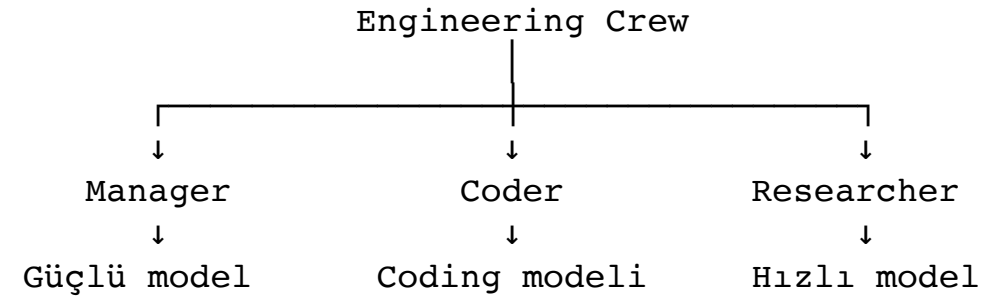
```
Career Agent
    ↓
MCP
    ↓
GitHub API
```

mantığıyla aynı düşüncedir.

37. Multiple LLMs

CrewAI'de bütün agent'ların aynı LLM'i kullanması gerekmez.

Mesela:



Örneğin manager için güçlü reasoning modeli seçebiliriz.

Basit araştırma agent'ı için daha ucuz ve hızlı model seçebiliriz.

Coder için kod konusunda başarılı başka model kullanabiliriz.

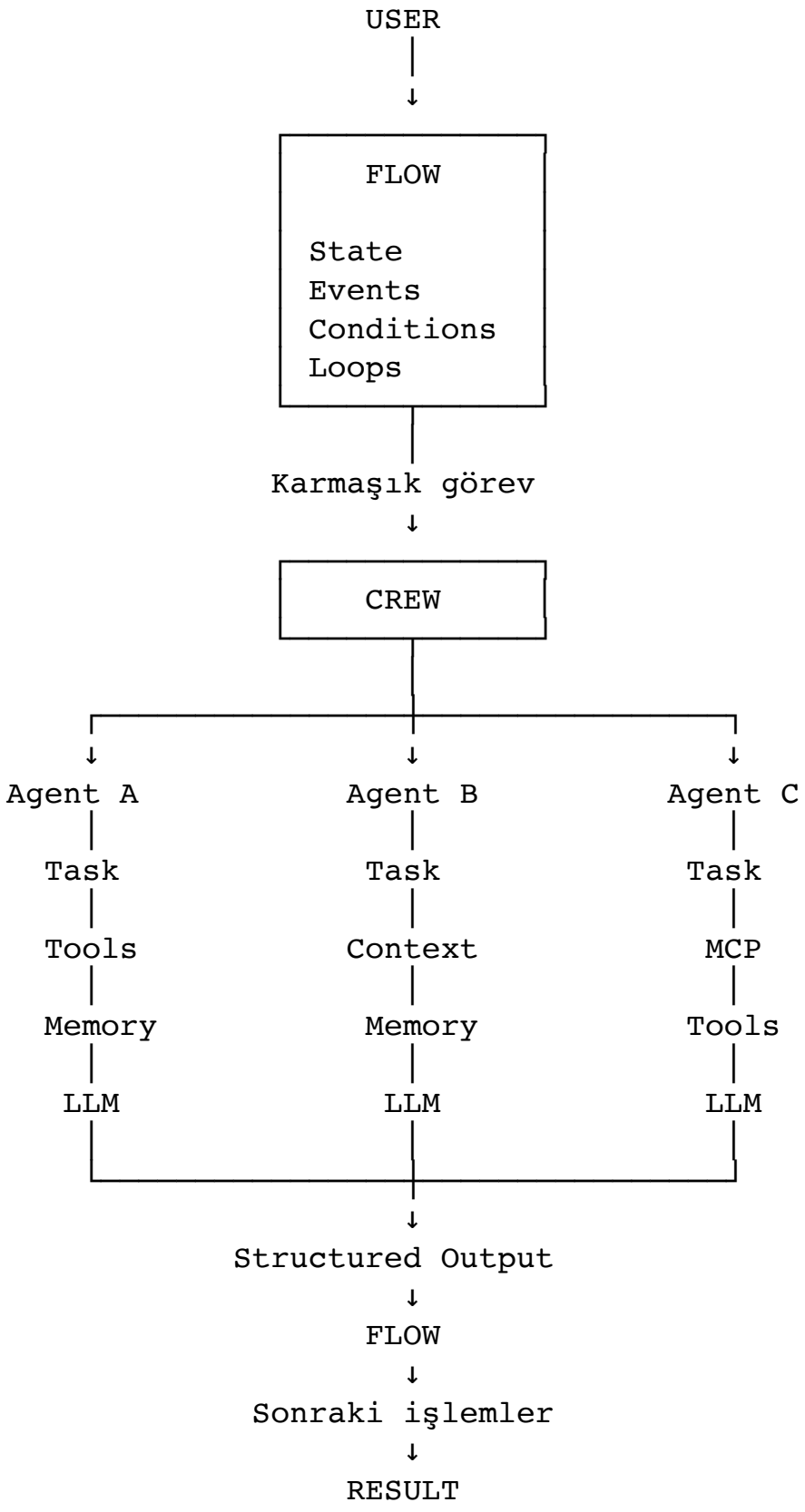
Böylece:

```
Kalite
  +
Hız
  +
Maliyet
```

arasında denge kurabiliriz.

38. CrewAI'de tüm yapı nasıl birleşiyor?

Şimdi bütün parçaları tek sistemde görelim:



39. Kursun gün gün ilerleyişi

Şimdi neden konuların bu sırada olduğunu da anlayabiliriz.

Day 1

Temeli öğreniyoruz:

```
CrewAI
↓
Agent
↓
Task
↓
Crew
↓
YAML
↓
crew.py
↓
main.py
```

Ve ilk **Debate Crew**'ü oluşturuyoruz.

Day 2

Agent'a dış dünya ekliyoruz:

```
Tools
+
Context
+
Serper Search
```

ve **Financial Research Crew** yapıyoruz.

Day 3

Sistemi profesyonelleştiriyoruz:

```
Structured Output
+
Pydantic
+
```

```
Custom Tools
+
Memory
+
Hierarchical Process
```

ve **Stock Picker Crew** oluşturuyoruz.

Day 4

Agent'a gerçek eylem yaptırıyoruz:

```
Coding Agent
+
Custom Sandbox Tool
+
Docker
+
Code Execution
```

Day 5

Bütün parçaları büyük bir multi-agent sisteminde birleştiriyoruz:

```
4 AI Agents
+
Tasks
+
Crew
+
MCP
+
UV Sandbox
+
Multiple LLMs
```

ve bir **AI Engineering Team** oluşturuyoruz.

40. En önemli kavramların tek cümlelik karşılığı

Agent: Belirli rolü ve amacı olan AI çalışanı.

Task: Agent'ın gerçekleştireceği görev.

Crew: Birlikte çalışan agent ekibi.

Flow: Uygulamanın genel iş akışını yöneten yapı.

Tool: Agent'ın dış dünyada işlem yapmasını sağlayan araç.

Context: Bir görevin diğer göreve aktardığı mevcut bilgi.

Memory: Agent sisteminin geçmişten sakladığı bilgi.

Process: Agent ve task'ların hangi düzenle çalışacağını belirleyen yöntem.

Sequential: Görevlerin önceden belirlenmiş sırayla çalışması.

Hierarchical: Manager agent'ın görevleri diğer agent'lara yöneterek dağıtması.

Structured Output: LLM çıktısının belirli bir veri yapısında üretilmesi.

Pydantic: Structured output'un Python tarafında şemasını ve doğrulamasını sağlayan yapı.

Sandbox: Agent'ın oluşturduğu kodun izole ortamda çalıştırılması.

MCP: Agent'ların dış araç ve veri kaynaklarına standart şekilde bağlanmasını sağlayan protokol.

20. Günün Büyük Akışı

```
CrewAI
  ↓
Flow + Crew
  ↓
Agent
  ↓
Task
  ↓
YAML
  ↓
crew.py + main.py
  ↓
Tools
  ↓
Context
  ↓
Structured Outputs
  ↓
```

```
Pydantic
  ↓
Custom Tools
  ↓
Memory
  ↓
Hierarchical Process
  ↓
Coding Agent
  ↓
Docker Sandbox
  ↓
MCP
  ↓
Multi-Agent Engineering Team
  ↓
Multiple LLMs
  ↓
Üretime yakın Multi-Agent AI sistemi
```

Bugünün ana fikri

CrewAI'de Flow uygulamanın kontrollü iş akışını yönetirken, Crew karmaşık işleri uzmanlaşmış Agent'lara dağıtır; Agent'lar Task'ları Tools, Context ve Memory yardımıyla gerçekleştirir, Process onların nasıl birlikte çalışacağını belirler ve ilerleyen aşamalarda Structured Outputs, Docker Sandbox, MCP ve farklı LLM'ler eklenerek daha güçlü ve gerçekçi multi-agent sistemleri oluşturulur.